

Manual breve de activación de características y reutilización de apps en Django

Contents

1. Instrucciones para activar y desactivar características	1
Objetivo	1
Pasos básicos	1
Ejemplo	2
Activar una variante	2
Desactivar una característica	2
Recomendación	2
2. Recomendaciones para reutilizar apps entre proyectos Django	3
3. Instrucciones para crear un PDF con este manual	4
Opción 1: desde Visual Studio Code	4
Opción 2: desde un navegador	4
Recomendación	4
Resumen	5

1. Instrucciones para activar y desactivar características

Objetivo

Permitir habilitar o deshabilitar funciones del proyecto sin modificar de forma permanente el código principal.

Pasos básicos

1. Definir las características en un archivo de configuración, por ejemplo `config_product.py`.
2. Asignar valores como `True` o `False` para cada característica.
3. Seleccionar la variante activa mediante la variable de entorno `PRODUCT_VARIANT`.
4. Leer esa configuración en `settings.py`.
5. Usar condiciones en las vistas, middleware o plantillas para activar o desactivar la funcionalidad.

Ejemplo

```
PRODUCT_A = {  
    "ENABLE_REPORTS": True,  
    "ENABLE_JWT": True,  
}  
  
PRODUCT_B = {  
    "ENABLE_REPORTS": False,  
    "ENABLE_JWT": False,  
}
```

Activar una variante

```
$env:PRODUCT_VARIANT="A"  
python manage.py runserver
```

Desactivar una característica

Si una característica debe quedar fuera de una variante, se asigna False en la configuración correspondiente.

Recomendación

Mantener las características en un solo lugar centralizado para evitar dispersión de condiciones en el código.

2. Recomendaciones para reutilizar apps entre proyectos Django

1. Separar la lógica de negocio de la interfaz

Mantener las vistas, formularios y modelos organizados de forma clara para poder migrarlos a otro proyecto con menos cambios.

2. Crear apps con responsabilidad única

Cada app debe enfocarse en una función concreta, por ejemplo: autenticación, usuarios, reportes o pagos.

3. Usar configuraciones parametrizadas

Evitar hardcodear valores fijos. Preferir variables de entorno o archivos de configuración.

4. Mantener dependencias mínimas

Una app reutilizable debe depender solo de lo necesario y no de lógica específica de un proyecto concreto.

5. Documentar la app

Incluir información sobre sus modelos, vistas, formularios y variables de configuración.

6. Aprovechar la arquitectura modular

Si una app funciona bien en un proyecto, puede integrarse en otro siempre que se ajusten sus configuraciones y dependencias.

7. Probar la app de forma independiente

Realizar pruebas antes de moverla a otro proyecto para verificar que funciona correctamente.